Guide de préparation — une démo MaaS reproductible

1. Le résultat attendu

Un administrateur publie un vrai modèle public depuis Hugging Face. Une équipe reçoit une souscription et une autorisation. Un data scientist crée sa clé personnelle et utilise une API compatible OpenAI. L'administrateur retrouve les tokens et les appels dans OpenShift AI.

Le fil conducteur tient en une phrase : **« Je déploie une fois, je donne un accès gouverné à une équipe, puis je mesure son usage. »**

Hugging Face fournit les fichiers du modèle. vLLM exécute l'inférence sur le GPU OpenShift. MaaS contrôle l'accès et le quota à la gateway. Une API de fournisseur externe constitue un autre scénario, avec des identifiants et un routage différents ; elle n'est pas utilisée ici.

La feuille `FEUILLE-DEMO.md` est le support à garder ouvert pendant la présentation. Ce document sert à préparer et reconstruire l'environnement.

2. Prérequis d'une nouvelle sandbox

La cible validée est OpenShift 4.20.42 avec OpenShift AI 3.5.2. Prévoir un compte cluster-admin, l'accès Internet aux registres Red Hat et Hugging Face, une StorageClass par défaut et au moins un GPU NVIDIA libre. Le modèle utilise 1 CPU demandé / 4 maximum, 4 GiB de RAM demandés / 8 maximum et 1 GPU.

Les opérateurs RHOAI, GPU et les dépendances de l'inférence llm-d doivent être disponibles. Le kit installe les dépendances MaaS et observabilité via les catalogues : Red Hat Connectivity Link (canal stable-1.4), Cluster Observability Operator et OpenTelemetry. Il ne transforme pas un cluster OpenShift vierge en une installation RHOAI complète.

Sur le poste : `oc`, Bash, Python 3 avec PyYAML, `curl`, `envsubst`, `make`, et OpenSSL ou htpasswd. Installer PyYAML dans un environnement Python si nécessaire. `make pdf` demande aussi `markdown` et Google Chrome/Chromium.

```
oc login --server=<API_DE_LA_SANDBOX>  # connexion selon le fournisseur d'identité
oc whoami
oc whoami --show-server
oc auth can-i '*' '*' --all-namespaces
make validate
```

La validation vérifie les catalogues et les schémas. Une CRD absente avant préparation est signalée comme SKIP ; cela ne prouve pas que l'installation est terminée. Toute erreur réelle doit être corrigée avant la démonstration.

Le domaine et le certificat sont découverts sur le cluster. Le kit cible les singletons `default-dsc` et `default-dsci` et le tenant MaaS `default-tenant`. Une sandbox utilisant d'autres noms doit être adaptée avant exécution.

3. Choisir le mode de préparation

A. Présenter toutes les créations en direct

```
make prepare
make endpoints
```

Cette commande prépare MaaS, PostgreSQL, la gateway, la collecte des métriques, le dashboard de démo, un profil matériel `MaaS - 1 GPU` et l'identité de consommation. Elle ne crée pas le projet ni le modèle. Sur une sandbox déjà utilisée, elle conserve l'existant : elle ne remet pas automatiquement les écrans à vide.

Ensuite suivre la feuille de démo : créer le projet, déployer le modèle, créer la souscription et l'autorisation, puis la clé utilisateur.

B. Reconstruire tout pour répéter rapidement

```
make demo
make endpoints
```

Le script s'arrête à la première erreur. Après correction, relancer la même commande. Les ressources ont des noms fixes et sont appliquées de manière déclarative. Les mots de passe existants et la base des clés ne sont pas régénérés à chaque relance. L'observabilité ne redémarre plus systématiquement le dashboard operator.

La création d'une clé dans l'interface reste le geste utilisateur à présenter. Le kit n'invente pas de clé et ne la substitue pas par le token admin.

Durée et cache

Le premier démarrage dépend du téléchargement des images et des opérateurs. Sur cette sandbox, les poids du petit Qwen ont été copiés en quelques secondes, mais l'image vLLM de près de 18 Go a demandé environ 2 min 30, puis le moteur a initialisé le GPU. Prévoir une marge de préparation de 30 à 60 minutes sur une sandbox neuve ; ce n'est pas une durée garantie.

Sur la même sandbox, une relance sans changement conserve le pod et les images. Le cache de la sandbox précédente ne survit pas à sa destruction. Le kit attend un rollout réellement terminé ; un ancien statut Ready ou un pod simplement Running ne suffit pas.

4. Les identités

`make prepare` détecte le fournisseur d'identité OpenShift. Il peut ajouter `demo-user` à un HTPasswd existant, ou utiliser le Keycloak intégré d'une sandbox dédiée. Pour un fournisseur externe, créer le compte selon le processus de cet IdP et l'associer au groupe OpenShift `demo-team` ; adapter l'étape identité du bootstrap, qui s'arrête explicitement sur ce cas. Si aucun IdP n'est configuré, `ADD_HTPASSWD_IDP=yes make prepare` autorise l'ajout du fournisseur HTPasswd de démonstration.

Le mot de passe généré est local : `secrets/demo-user.password`. Le répertoire est privé et exclu de Git. Un utilisateur HTPasswd existant n'autorise pas à retrouver son mot de passe ; restaurer le fichier local ou réinitialiser explicitement son mot de passe. Le kit ne change pas silencieusement le mot de passe d'un utilisateur existant.

Garder l'administrateur dans une **fenêtre normale** et ouvrir une **fenêtre de navigation privée / InPrivate** pour le data scientist. Dans cette fenêtre privée, ouvrir l'URL OpenShift AI de `make endpoints`, choisir le fournisseur de `demo-user` (**corp-demo** sur la sandbox validée), puis se connecter avec `demo-user` et le mot de passe du fichier local. Vérifier le compte affiché en haut à droite avant de montrer les souscriptions. Un profil de navigateur distinct convient aussi ; un simple nouvel onglet normal partage la session admin. Revenir à la fenêtre normale pour l'observabilité admin.

Les droits de consultation accordés pour le dashboard ne remplacent pas la souscription MaaS. Le manifeste de démonstration inclut des droits de découverte à l'échelle du cluster et d'administration du seul projet `maas-demo` pour les usages workbench ; ce n'est pas un modèle de cloisonnement de production.

5. Le modèle réel

Source : `hf://Qwen/Qwen2.5-0.5B-Instruct`. Le manifeste est `manifests/model/llm.yaml` et la publication `manifests/publish/model-ref.yaml`.

La mémoire de contexte est limitée à 2 048 tokens et vLLM utilise 60 % de la mémoire GPU. `--enforce-eager` évite une compilation longue pour cette démonstration. Le modèle de 0,5 milliard de paramètres illustre le parcours ; il ne sert pas à comparer la qualité des grands modèles.

```
oc get llminferenceservice demo-llm -n maas-demo
oc rollout status deployment/demo-llm-kserve -n maas-demo --timeout=900s
oc get maasmodelref demo-llm -n maas-demo
```

La publication peut attendre la gouvernance avant d'afficher son état global Ready. La disponibilité du moteur et l'association souscription/autorisation sont deux vérifications distinctes.

GPU occupé

Le script refuse le mode réel si aucun GPU n'est disponible. Il ne remplace pas automatiquement le LLM par un simulateur. Arrêter explicitement le modèle occupant le GPU depuis son bouton Stop, ou choisir une autre sandbox.

Sur la sandbox utilisée pour cette validation, avec accord de l'utilisateur, `my-first-model/llama-32-3b-instruct` a été arrêté via `serving.kserve.io/stop=true`. Pour le reprendre, arrêter d'abord `demo-llm`, puis retirer cette annotation ou utiliser Start dans l'interface. Ne démarrer qu'un modèle GPU à la fois sur ce cluster.

Répétition CPU

```
MODEL_MODE=simulator make demo
```

Ce mode charge `llm-simulator.yaml`. Dire explicitement : « Ici je répète les mécanismes MaaS avec des réponses synthétiques. » Revenir à `make demo` pour le vrai modèle.

6. La gouvernance et les clés

La souscription `demo-quota` associe `demo-team` au modèle `maas-demo/demo-llm`, avec une limite de 2 000 tokens par minute. L'autorisation `demo-access` permet à ce même groupe d'atteindre le modèle par la gateway. Les deux sont nécessaires.

L'interface propose **Create authorization policy** lors de la création d'une souscription. Si elle est cochée, elle crée une policy associée, généralement nommée `demo-quota-policy`. Pour conserver les noms exacts du kit, la feuille recommande de la décocher et de créer `demo-access` dans l'onglet suivant. Ne pas créer les deux policies en parallèle : une policy restante empêcherait de démontrer un refus d'accès.

```
make govern
make govern-verify
```

En tant que `demo-user` : **Gen AI studio** → **API keys** → **Create API key**. Choisir `demo-quota`, nommer la clé `demo-notebook`, choisir une expiration adaptée à la sandbox et enregistrer la valeur affichée une seule fois. `make save-key` permet sa capture locale. Une seconde clé liée à `demo-low-quota` sert exclusivement au test 429.

7. Appeler le modèle sans session admin

Dans un terminal utilisateur ou un workbench, disposer uniquement de `scripts/client.py`, de `curl`, de Python et du fichier de clé. Copier l'URL affichée par `make endpoints` depuis la préparation admin.

```
export MAAS_URL=https://maas.apps.<domaine-de-la-sandbox>
python3 scripts/client.py --key-file secrets/demo-user.apikey
```

Le client découvre les modèles accessibles dans `/v1/models`, sélectionne celui du projet de démo, puis appelle `/v1/chat/completions`. Il affiche le statut HTTP, la réponse et les tokens. Il ne nécessite aucun droit Kubernetes. Le secret est transmis à curl par l'entrée standard, pas dans les arguments affichables des processus.

```
make traffic
make quota-test
```

Le petit quota est volontairement inférieur à la consommation d'un appel. Le premier appel peut réussir, puis le suivant être refusé : la comptabilisation tient compte des tokens de la réponse. Un quota de tokens n'est pas une limite stricte sur le nombre d'appels ni une facturation.

8. L'observabilité à montrer

Admin : **Observe & monitor** → **Dashboard** → **MaaS démo**.

Cet onglet est une extension Perses fournie par le dépôt. Il est intégré au dashboard OpenShift AI et utilise les vrais compteurs Limitador collectés par Prometheus. Il montre les tokens de `demo-quota`, les appels autorisés, les refus du petit quota et une courbe de consommation par modèle/souscription sur des fenêtres de cinq minutes.

Les trois nombres sont des compteurs depuis le démarrage de Limitador, indépendants de la période de graphe sélectionnée. Ils repartent de zéro au redémarrage. La courbe utilise `increase`, donc une estimation Prometheus, susceptible d'extrapolation. Attendre 30 à 60 secondes après les appels, puis Refresh. En absence de série, l'interface indique No data ; ne pas l'interpréter comme une mesure de zéro.

L'onglet natif **Usage** de cette build demande des métriques suffixées `_total` et un label `user`. Les séries observées pour notre démo s'appellent `authorized_hits`, `authorized_calls`, `limited_calls` et portent `subscription`, sans `user`. Le dashboard du kit interroge ces séries réelles sans inventer de

données par utilisateur. Le fichier `demo-dashboard.json` devra être adapté si le schéma change dans une autre version.

```
make usage
```

Cette commande admin lit les compteurs à travers un port-forward authentifié et sauvegarde `evidence/usage.json`. Comparer l'augmentation de `authorized_hits` aux `total_tokens` des réponses. Ce suivi est destiné à la démonstration et au showback, pas à une facture financière.

Le test optionnel `make show-denials` retire temporairement `demo-team` des bénéficiaires de la policy, attend un refus, restaure les bénéficiaires exacts puis vérifie le retour en 200. Il attend jusqu'à deux minutes par phase pour laisser passer le cache de 60 secondes. Ne pas le lancer en parallèle de `make demo`. Sur cette build, supprimer complètement la policy a laissé une règle obsolète dans la gateway : ne pas utiliser cette suppression comme preuve de révocation. Ce défaut observé est détaillé dans `docs/VALIDATION.md`.

9. Rejouer et diagnostiquer

```
make demo                   # remettre l'état déclaratif complet
make live-reset CONFIRM=yes  # retire modèle/publication/gouvernance de démo
```

Le reset conserve le projet, la plateforme et la base des clés. Il ne supprime aucun PVC partagé ni gateway. Les clés existantes peuvent redevenir utilisables si la même souscription est recréée : les révoquer explicitement si l'objectif est de repartir sans aucune clé. Pour montrer aussi la création d'un projet vide, privilégier une nouvelle sandbox.

Pending GPU : vérifier les pods demandant `nvidia.com/gpu` et arrêter explicitement le modèle concurrent. **ImagePullBackOff :** vérifier l'accès au registre et le pull secret. **403 :** vérifier la clé, son expiration, le groupe, la souscription et la policy. **429 :** attendre la nouvelle fenêtre ou utiliser la clé principale. **Catalogue vide :** vérifier que le projet possède le label `maas.opendatahub.io/gateway-access=true` et que MaaSModelRef est publié.

TLS intermittent : vérifier les endpoints du Service de la gateway. Sur la sandbox historique, deux Deployments issus d'un changement de GatewayClass partageaient le même sélecteur ; l'ancien recevait encore du trafic. Il a été réduit à zéro. Cette migration n'existe pas normalement sur une sandbox neuve. Le client réessaie uniquement les échecs de handshake TLS, avant envoi d'une requête HTTP ; il ne répète pas automatiquement une inférence dont la réponse serait perdue.

10. Sources et périmètre de validation

- Red Hat : gouverner les accès LLM avec MaaS, 3.5
- Red Hat : observabilité OpenShift AI
- Modèle Qwen public
- API MaaS upstream

Les observations de cette sandbox, les essais réellement effectués et les limites sont consignés dans `docs/VALIDATION.md`. Une relance réussie sur la sandbox existante ne remplace pas un test d'installation intégrale sur une seconde sandbox neuve.